

**VIVEKANANDA INSTITUTE OF PROFESSIONAL STUDIES –
TECHNICAL CAMPUS**

**VIVEKANANDA SCHOOL OF INFORMATION
TECHNOLOGY**



योग: कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

**Machine Learning with Python Lab
BCAP 311**

**Bachelor of Computer Applications
(BATCH: 2023-2026)**



**Affiliated to
Guru Gobind Singh Indraprastha University, Delhi**

SUBMITTED TO:

Dr. Mamta Madan
VSIT, VIPS

SUBMITTED BY:

Name: Abhay
Enrollment No. :08117702023
BCA – V (B)

Index

S. no	Name	Date	Signature
1.	Extract one dataset from Kaggle. Explore its first 5 records and Last 5 records.		
2.	Remove the missing data from the dataset. Apply various preprocessing steps.		
3.	From the same dataset, change the format of one column and implement the same.		
4.	Perform all the important functions on IRIS dataset. Preprocess the IRIS dataset		
5.	Implement Linear Regression on two random Arrays of numpy.		
6.	Implement Linear Regression On salary.CSV file discussed in class.		
7.	Implement Linear regression on some unknown datasets.		
8.	Implement logistic regression as done in class for the same data set.		
9.	Apply logistic regression on insurance dataset from Kaggle based on age and gender.		
10.	Take any dataset from Kaggle and implement Decision tree.		
11.	Implement naïve bayes classifier for covid, flu and fever database.		
12.	Implement support vector machines for the example discussed in the class.		
13.	Write a program to implement k-nearest neighbors (KNN) using any dataset after preprocessing.		
14.	Implement a classification on iris dataset using random forest.		
15.	Implement Ensemble classifier using any dataset. Do complete preprocessing and show the accuracy after implementing ensemble classifier.		
16.	Build an Artificial Neural Network (ANN) by implementing the Back propagation algorithm.		
17.	Implement K means clustering for a dataset.		
18.	Implement Principal Component Analysis for any problem.		

Program 1

// extract one dataset from Kaggle. Explore its first 5 records and Last 5 records.

```
import pandas as pd
```

```
# Example: Titanic dataset (make sure you've set up Kaggle API and downloaded the dataset)
```

```
df = pd.read_csv("titanic/train.csv")
```

```
# First 5 records
```

```
print("First 5 records:")
```

```
print(df.head())
```

```
# Last 5 records
```

```
print("\nLast 5 records:")
```

```
print(df.tail())
```

Output:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0

Program 2

// Remove the missing data from the dataset. Apply various preprocessing steps.

```
# remove the missing data from the dataset
```

```
df = df.dropna()
```

```
df.isnull().sum()
```

0

PassengerId	0
--------------------	---

Survived	0
-----------------	---

Pclass	0
---------------	---

Name	0
-------------	---

Sex	0
------------	---

Age	0
------------	---

SibSp	0
--------------	---

Parch	0
--------------	---

Ticket	0
---------------	---

```
# now checking duplicated rows
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
# Convert categorical 'Sex' column to numeric (male=0, female=1)
```

```
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
```

```
# Rename columns for better readability
```

```
df.rename(columns={'Pclass': 'PassengerClass'}, inplace=True)
```

Program 3

// From the same dataset, change the format of one column and implement the same.

```
df['Name'] = df['Name'].str.upper()
df[['Name']].head()
```

Name	
1	CUMINGS, MRS. JOHN BRADLEY (FLORENCE BRIGGS TH...
3	FUTRELLE, MRS. JACQUES HEATH (LILY MAY PEEL)
6	MCCARTHY, MR. TIMOTHY J
10	SANDSTROM, MISS. MARGUERITE RUT
11	BONNELL, MISS. ELIZABETH

```
# convert fare(float) -> int
df['Fare'] = df['Fare'].astype(int)
df[['Fare']].head()
```

Fare	
1	71
3	53
6	51
10	16
11	26

```
# (Optional) Save cleaned dataset
df.to_csv("titanic_cleaned.csv", index=False)
print("\nCleaned dataset saved as 'titanic_cleaned.csv'")
```

Cleaned dataset saved as 'titanic_cleaned.csv'

Program 4

// Perform all the important functions on IRIS dataset. Preprocess the IRIS dataset

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Load dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Replace numeric target with class names
df['species'] = df['species'].map({i: name for i, name in enumerate(iris.target_names)})

print("First 5 records:\n", df.head())
print("\nLast 5 records:\n", df.tail())

# 1. Check for missing values
print("\nMissing values:\n", df.isnull().sum())

# 2. Encode categorical column ("species")
label_enc = LabelEncoder()
df['species'] = label_enc.fit_transform(df['species'])

# 3. Scale numeric features
scaler = StandardScaler()
num_cols = df.drop('species', axis=1).columns
df[num_cols] = scaler.fit_transform(df[num_cols])

# Final processed dataset
print("\nProcessed Data:\n", df.head())
```

Output:

```
df.isnull().sum()
```

0

sepal length (cm) 0

sepal width (cm) 0

petal length (cm) 0

petal width (cm) 0

species 0

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 150 entries, 0 to 149

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	species	150 non-null	object

dtypes: float64(4), object(1)

memory usage: 6.0+ KB

Program 5

// Implement Linear Regression on two random Arrays of numpy.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Generate random data
np.random.seed(42) # reproducibility
x = np.random.rand(50) * 10
y = 3.2 * x + 5 + np.random.randn(50) * 2 # linear relation with noise

# Reshape x for sklearn (expects 2D)
x = x.reshape(-1, 1)

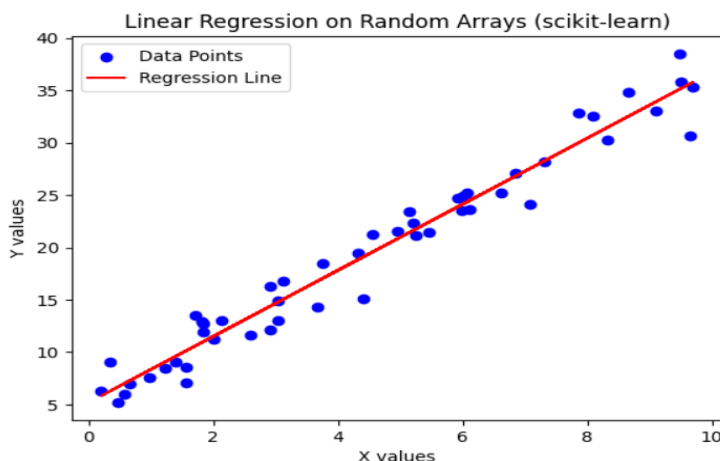
# Create and fit the model
model = LinearRegression()
model.fit(x, y)

# Predictions
y_pred = model.predict(x)

# Print coefficients
print("Intercept (b):", model.intercept_)
print("Slope (m):", model.coef_[0])

# Plot
plt.scatter(x, y, color='blue', label="Data Points")
plt.plot(x, y_pred, color='red', label="Regression Line")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```

Output:



▼ LinearRegression ⓘ ?
LinearRegression()

Program 6

// Implement Linear Regression On salary.CSV file discussed in class.

```
df1 = pd.read_csv("/content/Salary_Data.csv")
```

```
df1.head()
```

```
df1.shape
```

```
plt.scatter(df1['YearsExperience'], df1['Salary'])
```

```
X = df1[['YearsExperience']] # because of 1D
```

```
y = df1['Salary']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42, test_size=0.2)
```

```
from sklearn.preprocessing import StandardScaler
```

```
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train)
```

```
X_test = sc.transform(X_test)
```

```
y_pred = lr.predict(X_test)
```

```
print("Slope (Coefficient):", lr.coef_[0])
```

```
print("Intercept:", lr.intercept_)
```

```
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

```
print("R2 Score:", r2_score(y_test, y_pred))
```

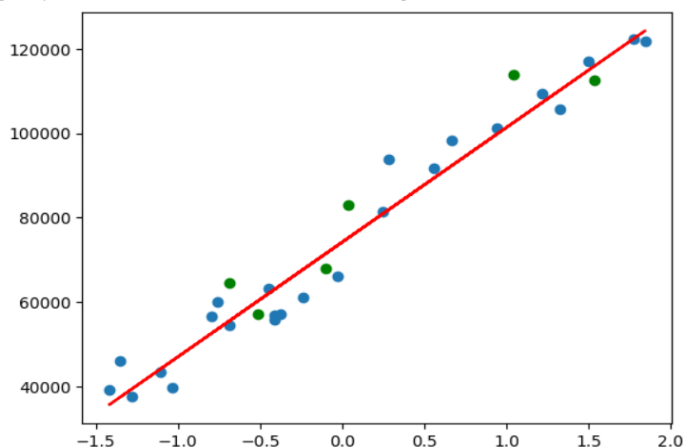
```
plt.scatter(X_train, y_train)
```

```
plt.scatter(X_test, y_test, color = 'g')
```

```
plt.plot(X_train, lr.predict(X_train), 'r')
```

Output:

[<matplotlib.lines.Line2D at 0x7d2aea0ba650>]



Slope (Coefficient): 27151.53742583369

Intercept: 74207.625

Mean Squared Error: 49830096.85590834

R² Score: 0.9024461774180498

Program 7

// Implement Linear regression on some unknown datasets.

```
from sklearn.datasets import fetch_california_housing

california = fetch_california_housing(as_frame=True)
df = california.frame

X = df.drop(columns=['MedHouseVal'])
y = df['MedHouseVal']

X_train, X_test, y_train, y_test = train_test_split(X,y , test_size=0.25, random_state=42)

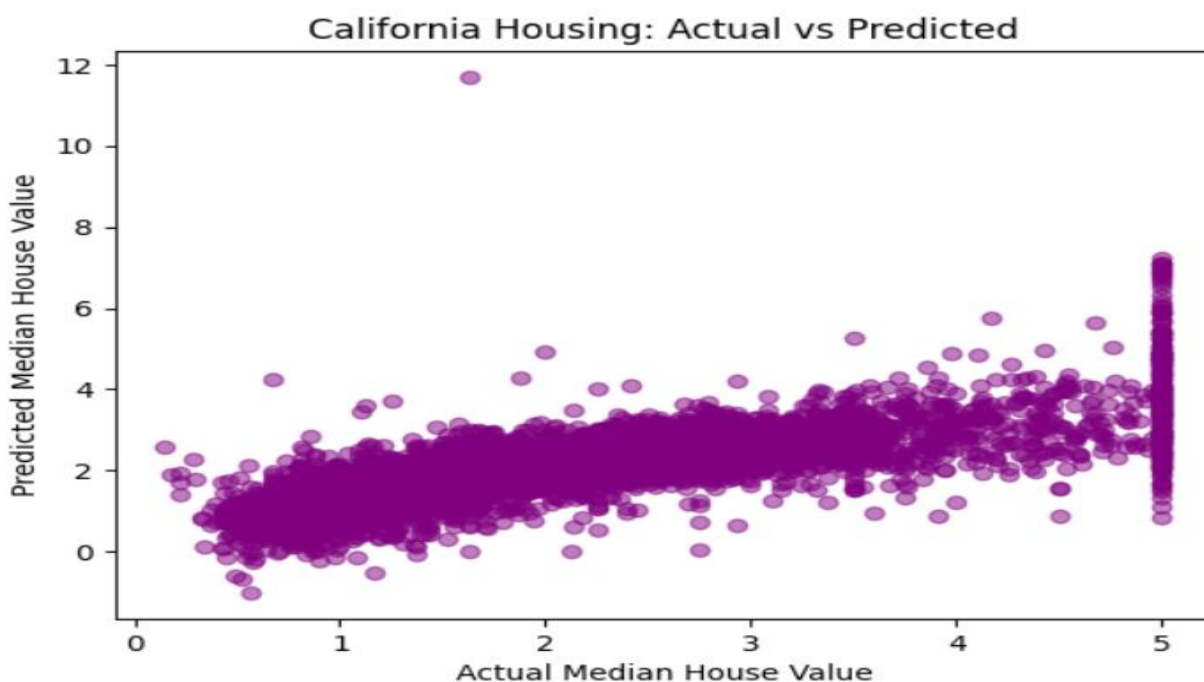
y_pred = lr.predict(X_test)

print("mean square error:",mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

plt.scatter(y_test, y_pred, alpha=0.5, color='purple')
plt.xlabel("Actual Median House Value")
plt.ylabel("Predicted Median House Value")
plt.title("California Housing: Actual vs Predicted")
plt.show()
```

Output:

```
mean square error: 0.5411287478470685
R2 Score: 0.5910509795491354
```



Program 8

// Implement logistic regression as done in class for the same data set.

```
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

df = pd.read_csv('/content/Salary_Data.csv')

# Convert continuous salary into a binary class (1 = High Salary, 0 = Low Salary)
threshold = df['Salary'].median()
df['HighSalary'] = (df['Salary'] > threshold).astype(int)

X = df[['YearsExperience']]
y = df['HighSalary']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

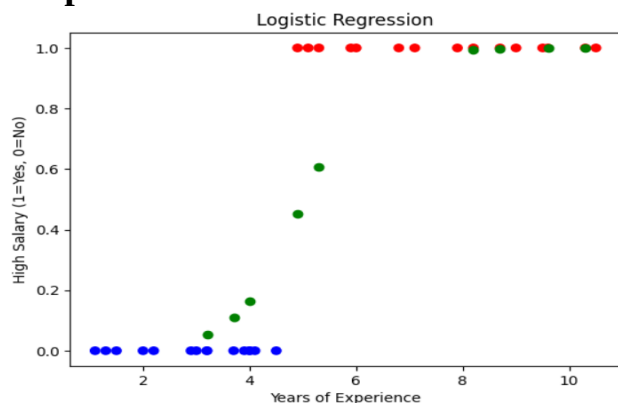
model = LogisticRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

plt.scatter(X, y, c=y, cmap='bwr')
plt.plot(X_test, model.predict_proba(X_test)[:,:1], 'go')
plt.xlabel("Years of Experience")
plt.ylabel("High Salary (1=Yes, 0=No)")
plt.title("Logistic Regression")
plt.show()
```

Output:



```
Classification Report:
              precision    recall  f1-score   support

     0       0.75         1.00         0.86         3
     1       1.00         0.83         0.91         6

 accuracy          0.89
 macro avg         0.88         0.92         0.88         9
 weighted avg      0.92         0.89         0.89         9
```

Program 9

//Apply logistic regression on insurance dataset from Kaggle based on age and gender.

```
df = pd.read_csv("/content/archive.zip")

# Convert charges into binary class: High (1) if above median, else Low (0)
threshold = df['charges'].median()
df['HighCharges'] = (df['charges'] > threshold).astype(int)

# Select features: age and sex (convert to numeric)
df['sex'] = df['sex'].map({'male': 0, 'female': 1})
X = df[['age', 'sex']]
y = df['HighCharges']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

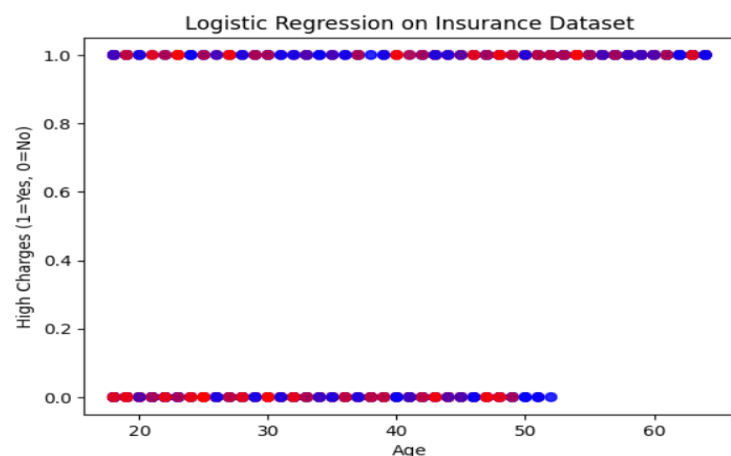
model = LogisticRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Scatter plot of age vs charges classification
plt.scatter(df['age'], df['HighCharges'], c=df['sex'], cmap='bwr', alpha=0.6)
plt.xlabel("Age")
plt.ylabel("High Charges (1=Yes, 0=No)")
plt.title("Logistic Regression on Insurance Dataset")
plt.show()
```

Output:



Classification Report:

	precision	recall	f1-score	support
0	0.74	0.70	0.72	207
1	0.70	0.74	0.72	195
accuracy			0.72	402
macro avg	0.72	0.72	0.72	402
weighted avg	0.72	0.72	0.72	402

Program 10

// Take any dataset from Kaggle and implement Decision tree.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, recall_score, f1_score
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = DecisionTreeClassifier(criterion="entropy", max_depth=5, random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

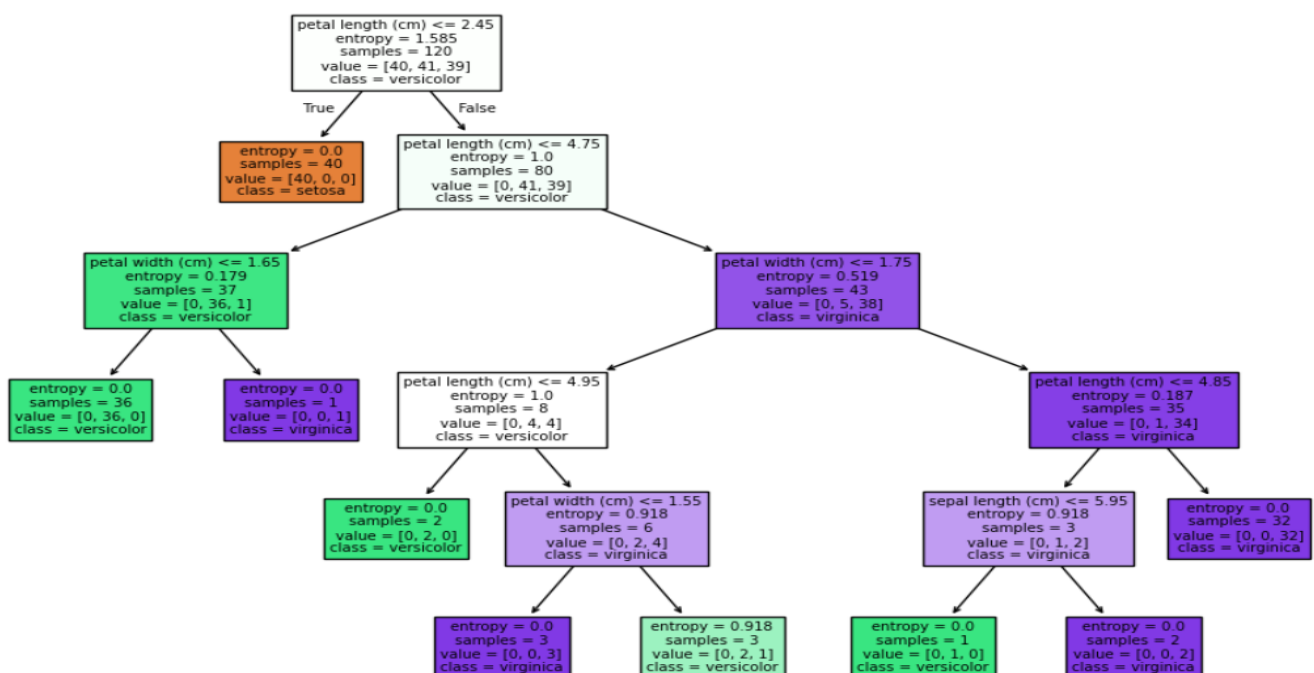
print ("Accuracy:", accuracy_score(y_test, y_pred))

import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(12,8))
plot_tree(clf, feature_names=iris.feature_names, class_names=iris.target_names,
filled=True)
plt.show()
```

Output:

Accuracy: 1.0



Program 11

// Implement naïve bayes classifier for covid, flu and fever database.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Synthetic dataset: symptoms vs disease
data = {
    "Cough": ["Yes", "Yes", "No", "Yes", "No", "Yes", "No", "Yes", "No", "No"],
    "SoreThroat": ["Yes", "No", "No", "Yes", "Yes", "No", "Yes", "No", "Yes", "No"],
    "Temperature":
["High", "High", "Normal", "High", "Normal", "High", "Normal", "Normal", "High", "Normal"],
    "Headache": ["Yes", "Yes", "No", "Yes", "No", "Yes", "Yes", "No", "Yes", "No"],
    "Disease":
["COVID", "COVID", "Fever", "Flu", "Fever", "COVID", "Flu", "Fever", "Flu", "Fever"]
}

# Encode categorical features numerically
for col in df.columns:
    df[col] = df[col].astype('category').cat.codes

X = df.drop(columns=['Disease'])
y = df['Disease']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

nb = CategoricalNB()
nb.fit(X_train, y_train)

# Predictions
y_pred = nb.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
#accuracy, list(zip(y_test, y_pred))
```

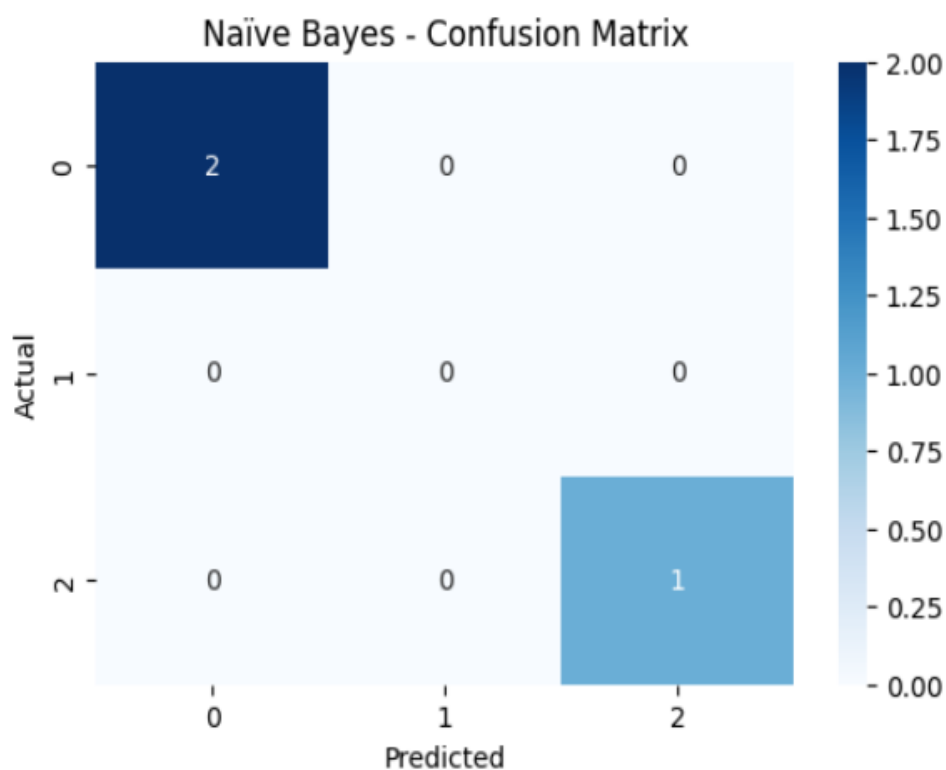
```

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred, labels=nb.classes_)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=nb.classes_, yticklabels=nb.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Naïve Bayes - Confusion Matrix")
plt.show()

```

Output:

Accuracy: 1.0



Program 12

// Implement support vector machines for the example discussed in the class.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

iris = datasets.load_iris()
X = iris.data[:, :2] # take only first two features for visualization
y = (iris.target != 0) * 1 # convert to binary: class 0 vs. others

# 2. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 3. Train a Support Vector Machine classifier
clf = SVC(kernel='linear', C=1.0)
clf.fit(X_train, y_train)

# 4. Evaluate accuracy
print("Test accuracy:", clf.score(X_test, y_test))

def plot_decision_boundary(clf, X, y):
    plt.figure(figsize=(8,6))
    # plot points
    plt.scatter(X[:,0], X[:,1], c=y, cmap=plt.cm.coolwarm, edgecolors='k')

    # plot decision boundary
    ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

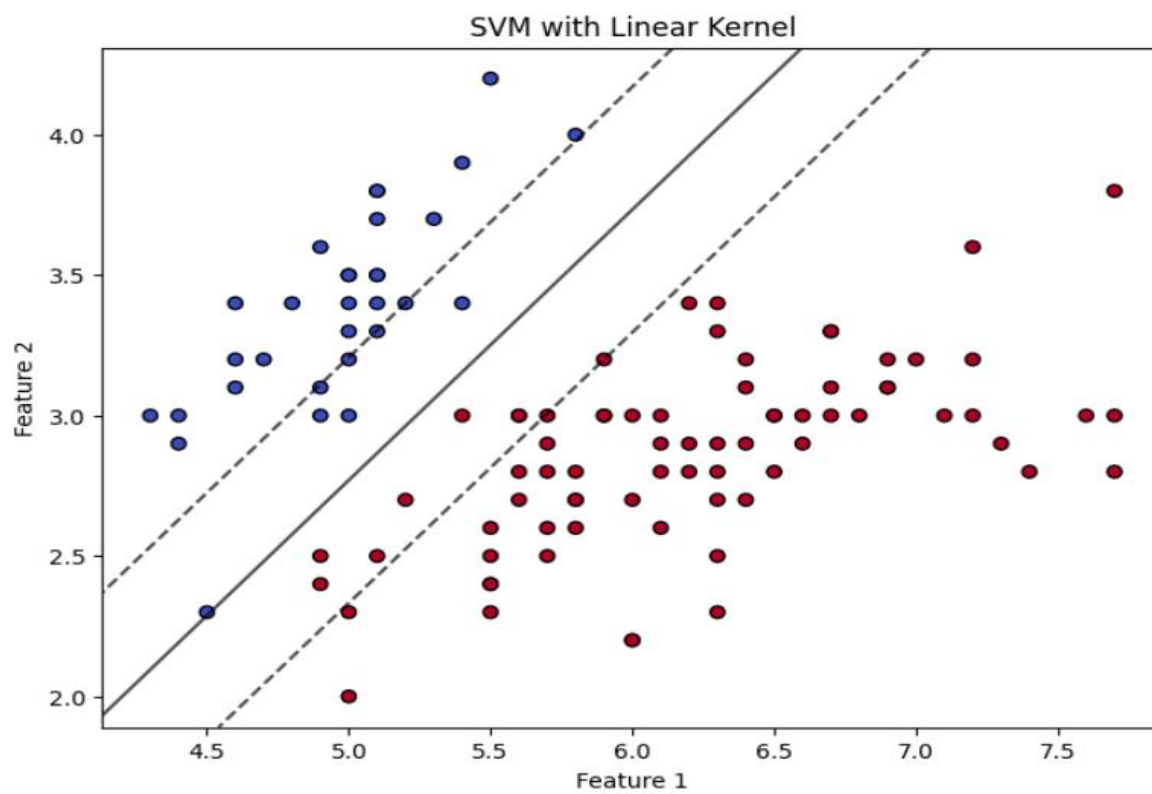
    xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], 200),
                          np.linspace(ylim[0], ylim[1], 200))
    Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # draw contours
    ax.contour(xx, yy, Z, colors='k', levels=[-1,0,1], alpha=0.7,
               linestyle=['--','-','--'])
    plt.title("SVM with Linear Kernel")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.show()
```

Output:

Test accuracy: 1.0

▼ SVC ⓘ ?
SVC(kernel='linear')



Program 13

// Write a program to implement k-nearest neighbors (KNN) using any dataset after preprocessing.

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X, y = iris.data, iris.target
class_names = iris.target_names

scaler = StandardScaler()
X = scaler.fit_transform(X)

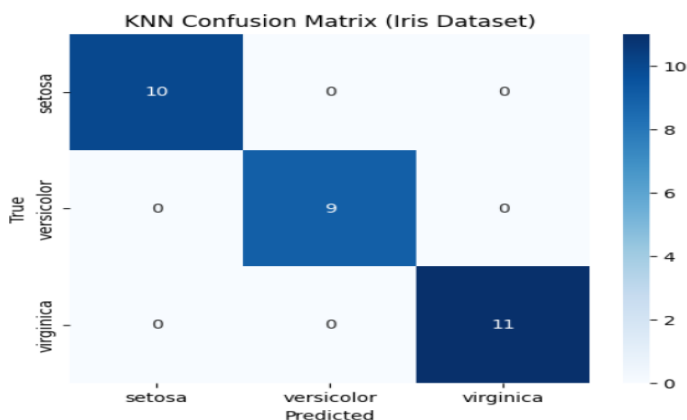
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("KNN Predictions:", y_pred)
print("True Labels:", y_test)
print(f"KNN Accuracy: {accuracy * 100:.2f}%")

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.title("KNN Confusion Matrix (Iris Dataset)")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

Output:



KNN Accuracy: 100.00%

Program 14

// Implement a classification on iris dataset using random forest.

```
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X, y = iris.data, iris.target
class_names = iris.target_names

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

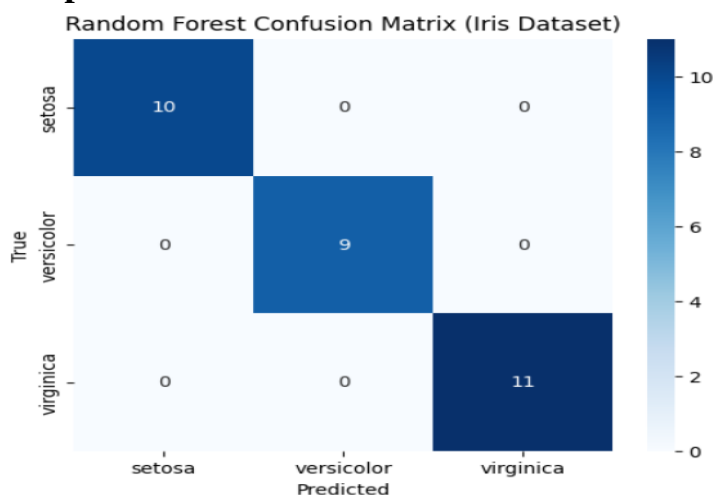
accuracy = accuracy_score(y_test, y_pred)

print(f"Random Forest Accuracy: {accuracy * 100:.2f}%")

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.title("Random Forest Confusion Matrix (Iris Dataset)")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

Output:



Random Forest Accuracy: 100.00%

Program 15

// Implement Ensemble classifier using any dataset. Do complete preprocessing and show the accuracy after implementing ensemble classifier.

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.neighbors import KNeighborsClassifier
from matplotlib import pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, confusion_matrix

# Load & Preprocess Data
iris = load_iris()
X, y = iris.data, iris.target

# Standardize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

log_clf = LogisticRegression(max_iter=200, random_state=42)
rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
knn_clf = KNeighborsClassifier(n_neighbors=5)

ensemble_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rf_clf), ('knn', knn_clf)],
    voting='hard'
)

ensemble_clf.fit(X_train, y_train)
y_pred = ensemble_clf.predict(X_test)

models = {
    "Logistic Regression": log_clf,
    "Random Forest": rf_clf,
    "KNN": knn_clf,
    "Ensemble": ensemble_clf
}
```

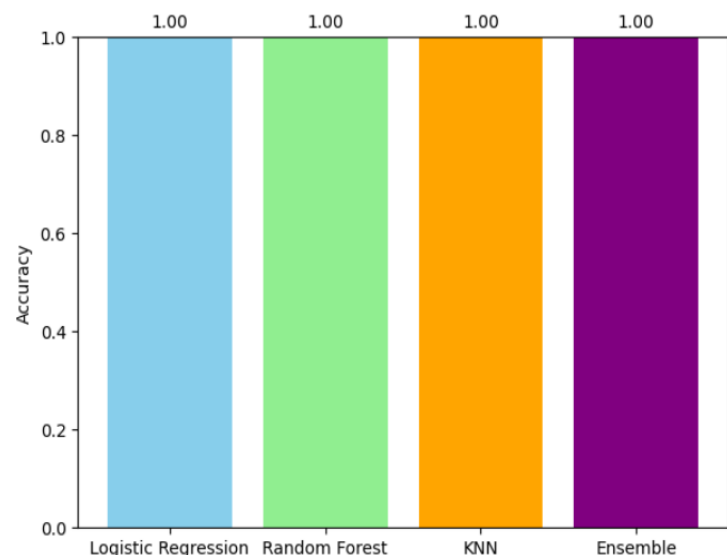
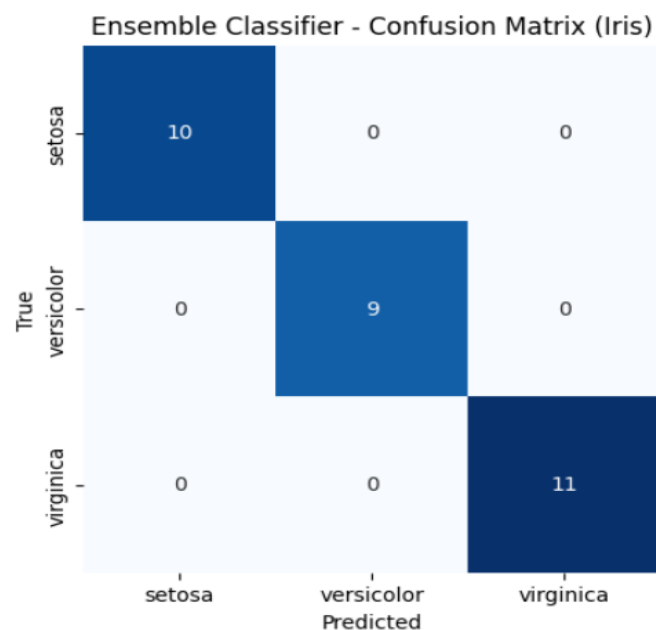
```
accuracy = accuracy_score(y_test, y_pred)
print("Ensemble Classifier Predictions:", y_pred)
print("True Labels:", y_test)
print(f"Ensemble Classifier Accuracy: {accuracy * 100:.2f}")
```

Plot Confusion Matrix for Ensemble

```
-----
plt.figure(figsize=(6, 5))
class_names = iris.target_names # Define class_names here
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.title("Ensemble Classifier - Confusion Matrix (Iris)")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```

Output:

```
Ensemble Classifier Predictions: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 2 2 2 2 0 0]
True Labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 2 2 2 2 0 0]
Ensemble Classifier Accuracy: 100.00%
```



Program 16

// Build an Artificial Neural Network (ANN) by implementing the Back propagation algorithm.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import pandas as pd

# Load the dataset
breast_cancer = load_breast_cancer()
X = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)
y = pd.Series(breast_cancer.target, name='target')

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert scaled arrays back to DataFrames for clarity (optional but good practice)
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_test.columns)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, BatchNormalization, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Initialize the Sequential model
model = Sequential()

# Add the input layer and the first hidden layer with Batch Normalization and Dropout
model.add(Dense(units=32, activation='relu', input_shape=(X_train_scaled.shape[1],)))
model.add(BatchNormalization())
model.add(Dropout(0.3))

# Add additional hidden layers with Batch Normalization and Dropout
model.add(Dense(units=16, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.3))

# Add the output layer
model.add(Dense(units=1, activation='sigmoid'))
```

```
# Configure the model for training
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
# Define Early Stopping callback
early_stopping = EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True)
```

```
# Train the model
history = model.fit(X_train_scaled, y_train, epochs=100, validation_split=0.2,
callbacks=[early_stopping])
```

Output:

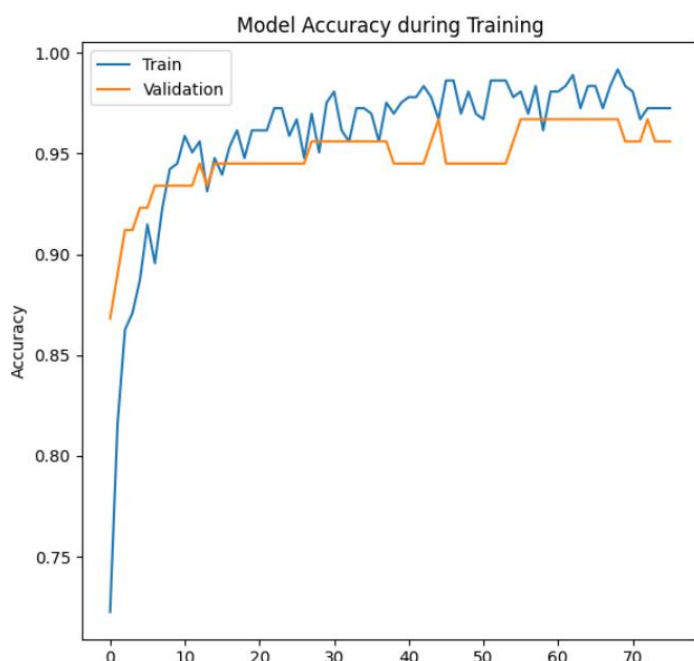
Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 32)	992
batch_normalization (BatchNormalization)	(None, 32)	128
dropout (Dropout)	(None, 32)	0
dense_4 (Dense)	(None, 16)	528
batch_normalization_1 (BatchNormalization)	(None, 16)	64
dropout_1 (Dropout)	(None, 16)	0
dense_5 (Dense)	(None, 1)	17

Total params: 1,729 (6.75 KB)

Trainable params: 1,633 (6.38 KB)

Non-trainable params: 96 (384.00 B)



Program 17

// Implement K means clustering for a dataset.

```
from sklearn.datasets import make_blobs
```

```
X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=0.5, random_state=42)
```

```
print("Shape of X:", X.shape)
```

```
print("Shape of y_true:", y_true.shape)
```

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
```

```
kmeans.fit(X)
```

```
y_kmeans = kmeans.predict(X)
```

```
plt.figure(figsize=(8, 6))
```

```
scatter = plt.scatter(X[:, 0], X[:, 1], c=y_kmeans)
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

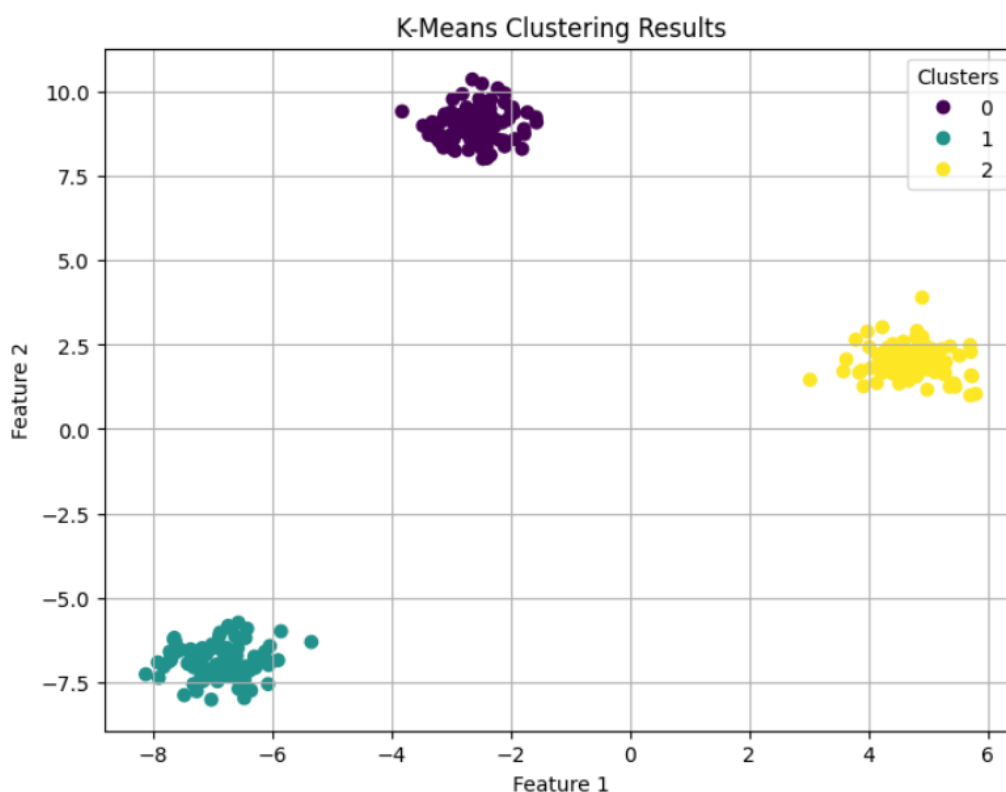
```
plt.title("K-Means Clustering Results")
```

```
legend = plt.legend(*scatter.legend_elements(), title="Clusters")
```

```
plt.grid(True)
```

```
plt.show()
```

Output



Program 18

// Implement Principal Component Analysis for any problem.

```
# Data Loading
import pandas as pd
wine_data_path = "https://docs.google.com/spreadsheets/d/e/2PACX-csv"
wine = pd.read_csv(wine_data_path)

# We need to remove duplicated rows and missing value rows
import pandas as pd

# Load the Wine Quality dataset
wine_data = pd.read_csv(wine_data_path)
print("Wine Data Shape :", wine_data.shape)
# Drop rows with missing values
wine_data = wine_data.dropna()

# Dropping Duplicates rows
wine_data.drop_duplicates(inplace=True)
print("Wine Data Shape (After Dropping-) :", wine_data.shape)

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Separate features (physicochemical properties) and target variable (type)
X = wine_data.drop('type', axis=1)
y = wine_data['type']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train a K-Nearest Neighbors (KNN) classifier without PCA
knn_no_pca = KNeighborsClassifier(n_neighbors=5)
knn_no_pca.fit(X_train_scaled, y_train)

# Predict the wine type on the testing set without PCA
y_pred_no_pca = knn_no_pca.predict(X_test_scaled)
```



```
# Evaluate the classification accuracy without PCA
accuracy_no_pca = accuracy_score(y_test, y_pred_no_pca)
print("KNN having features -", X_train_scaled.shape[1])
print("Classification Accuracy without PCA: {:.2f}%".format(accuracy_no_pca * 100))
```

```
KNN having features - 12
Classification Accuracy without PCA: 99.34%
```

```
# Apply PCA on the standardized features
pca = PCA(n_components=5)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

```
# Train a K-Nearest Neighbors (KNN) classifier with PCA
knn_with_pca = KNeighborsClassifier(n_neighbors=5)
knn_with_pca.fit(X_train_pca, y_train)
```

```
# Predict the wine type on the testing set with PCA
y_pred_pca = knn_with_pca.predict(X_test_pca)
```

```
# Evaluate the classification accuracy with PCA
accuracy_pca = accuracy_score(y_test, y_pred_pca)
```

```
print("KNN having features -", X_train_pca.shape[1])
print("Classification Accuracy with PCA: {:.2f}%".format(accuracy_pca * 100))
```

```
KNN having features - 5
Classification Accuracy with PCA: 98.87%
```

Output

